



TRAINEE DADOS EDUARDO QUAGGIO

15/07/2026

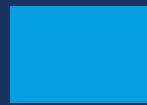
SEMANA 1

O foco dessa semana foi revisar alguns conceitos básicos de estatística e estudar as bibliotecas NumPy e Pandas através do curso de Data Analytics da Rocketseat Estatística

Foi realizada uma revisão dos conceitos:

- Média
- Mediana
- Amplitude
- Desvio padrão
- Quartis
- Entre outros

PROBLEMA:



SOLUÇÃO:



SEMANA 1 (PRÁTICAS)

NUMPY

PROBLEMA 217 - CONTAINS DUPLICATE

DADO UM VETOR DE NÚMEROS INTEIROS, DETERMINAR SE EXISTE ALGUM ELEMENTO REPETIDO. O OBJETIVO NÃO ERA ENCONTRAR A SOLUÇÃO MAIS OTIMIZADA, MAS SIM PROVAR QUE ESSA BIBLIOTECA PODE RESOLVER PROBLEMAS APARENTEMENTE COMPLEXOS EM POUCAS LINHAS DE FORMA INTUITIVA.

```
IMPORT NUMPY AS NP
```

```
CLASS SOLUTION:
```

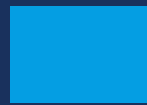
```
    DEF CONTAINSDUPLICATE(SELF, NUMS: LIST[INT]) -> BOOL:
```

```
        NUMS = NP.ARRAY(NUMS)
        TAMANHO_ORIGINAL = NUMS.SIZE
```

```
        NUMS = NP.UNIQUE(NUMS)
        TAMANHO_NOVO = NUMS.SIZE
```

```
        IF (TAMANHO_NOVO == TAMANHO_ORIGINAL):
            RETURN FALSE
        ELSE:
            RETURN TRUE
```

PROBLEMA:



SOLUÇÃO:



SEMANA 1 (PRÁTICAS)

PANDA

IMPORTAÇÃO DE UM ARQUIVO CSV DO DATASET IRIS EM UM PRJETO DO GOOGLE COLAB

```
IMPORT PANDAS AS PD
URL = 'HTTPS://RAW.GITHUBUSERCONTENT.COM/UIUC-CSE/DATA-FA14/GH-PAGES/DATA/IRIS.CSV'

DF = PD.READ_CSV(URL)

PRINT(DF)

DF.TO_CSV('IRIS.CSV', INDEX = FALSE)
```

SEMANA 2

Nessa segunda semana foram estudadas as aulas de Pandas e coleta de dados da Rocketseat

Aprofundando em Pandas

As aulas foram assistidas por completo na qual foi possível aprender todo o básico sobre a biblioteca, incluindo:

- Head, Shape, Info, Describe
- Loc e Illoc
- Ordenação
- Filtragens Apply e Map

Coleta de dados

Uso de SQL em Python

SEMANA 2 (PRÁTICAS)

8. SQL + Queries

```
df2 = pd.read_csv('/content/drive/MyDrive/ipl/ipl_comprehensive_dataset.csv')

# adicionando match number no primeiro dataframe para ser possível fazer join no futuro
df = pd.concat([df, df2['match_number']], axis = 1)

conn = sqlite3.connect(':memory:')

df.to_sql ("data", conn, index = False , if_exists = "replace")

df2.to_sql ("match", conn, index = False , if_exists = "replace")

# Linhas em que Kolkata Knight Riders ganharam
pd.read_sql_query("SELECT winner FROM data WHERE winner = 'Kolkata Knight Riders'", conn)
```

COMANDOS SQL EM PYTHON

LOC E REMOÇÃO DE DUPLICATAS

6. Loc

```
print(df.loc[200])
```

Mostrar saída oculta

7. Removendo duplicatas

```
# tamanho original
print(f"tamanho original: {df.shape}")

# removeu 25 linhas com valores nulos
df = df.dropna()

# tamanho novo
print(f"tamanho novo: {df.shape}")
```

... Mostrar saída oculta

SEMANA 3

A semana 3 teve como foco o final da sessão de Data Analytics e o início do aprendizado em machine learning.

Tratamento de dados

Nessa semana aprendi conceitos muito importantes sobre como identificar e lidar com duplicatas, valores ausentes e outliers. Além de outros conceitos como:

- Plotagem de gráficos
- Correlação + Heatmap
- Normalização
- One hot e Label Encoding

SEMANA 3 (PRÁTICA)

Primeiro teste com machine learning

O objetivo foi tratar os dados e criar um código para prever o custo do seguro de saúde, com base na idade, gênero, IMC, quantidade de filhos, se é fumante e a região.

TRATANDO OS DADOS

Removendo duplicatas e checando por valores ausentes

```
# Removendo duplicatas
df.drop_duplicates(keep = 'first', inplace = True)

# Sem valores ausentes
df.isna().sum()
```

Mostrar saída oculta

TRATANDO OUTLIERS

```
q1 = df["charges"].quantile(0.25)
q3 = df["charges"].quantile(0.75)
iqr = q3 - q1

li = q1 - 1.5 * iqr
ls = q3 + 1.5 * iqr

outliers = df[(df["charges"] >= ls) | (df["charges"] <= li)]

print(f"outliers:{len(outliers)}")
print(f"tamanho total:{len(df)}")

# Porcentagem muito alta, pelo gráfico poucos desses valores devem ser outliers
print(f"porcentagem: {100 * len(outliers)/len(df)}%")

outliers_acima = df[(df["charges"] >= ls)]

print(f"outliers acima:{len(outliers_acima)}")

df5 = df[(df["charges"] >= 50000)]
print(f"maior que 50000:{len(df5)}")

outliers:139
tamanho total:1337
porcentagem: 10.396409872849663%
outliers acima:139
maior que 50000:7
```

TRATAMENTO FINAL

Tratamentos finais

```
# One hot encoding
df = pd.get_dummies(df, columns = ['sex', 'smoker', 'region'])
df.head(10)
```

Mostrar saída oculta

```
# Normalização
df['age'] = (df['age'] - df['age'].min()) / (df['age'].max() - df['age'].min())
df['bmi'] = (df['bmi'] - df['bmi'].min()) / (df['bmi'].max() - df['bmi'].min())
df['children'] = (df['children'] - df['children'].min()) / (df['children'].max() - df['children'].min())
```

APLICANDO REGRESSÃO LINEAR

```
x = df.drop('charges', axis = 1)
y = df['charges']

x_train, x_test, y_train, y_test = train_test_split(x, y, train_size = 0.75, random_state = 42)

model = LinearRegression()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

mse = mean_squared_error(y_test, y_pred)

print(f'MSE: {mse}')

r2_score = r2_score(y_test, y_pred)

print(f'R2 Score: {r2_score}')
```

MSE: 33941605.94459926

R2 Score: 0.7402490043111913

SEMANA 4

Como sugerido, nessa semana começou-se os procedimentos no dataset de House Prices.

Além disso, também foram aprendidos os seguintes conceitos e como implementá-los em Python:

- K-fold
- Pipeline de pré-processamento
- Teste de normalidade
- Salvamento de modelos e API

SEMANA 4 (PRÁTICA)

```
# Dataset
df_train = pd.read_csv("train.csv")
df_train.head()
```

Mostrar saída oculta

```
df_train.info()
```

Mostrar saída oculta

```
# Removendo duplicatas e a coluna ID (inútil para ajudar na dedução do SalePrice)
df_train.drop_duplicates()
df_train.drop("Id", axis=1, inplace=True)

# Valores nulos por coluna
nulos = df_train.isna().sum()
print(nulos[nulos > 0])
```

Mostrar saída oculta

SEMANA 4 (PRÁTICA)

VALORES NULOS

LotFrontage	259
Alley	1369
MasVnrType	872
MasVnrArea	8
BsmtQual	37
BsmtCond	37
BsmtExposure	38
BsmtFinType1	37
BsmtFinType2	38
Electrical	1
FireplaceQu	690
GarageType	81
GarageYrBlt	81
GarageFinish	81
GarageQual	81
GarageCond	81
PoolQC	1453
Fence	1179
MiscFeature	1406
dtype: int64	

DADOS AUXILIARES

```
# Dados auxiliares para tomada de decisão
print(len(df_train[df_train["MasVnrArea"] == 0]))
print(len(df_train[df_train["TotalBsmtSF"] == 0]))
print(len(df_train[df_train["GarageArea"] == 0]))
print(len(df_train[df_train["PoolArea"] == 0]))
print(len(df_train[df_train["MiscVal"] == 0]))
```

861
37
81
1453
1408

SEMANA 5 (PRÁTICA)

Concluiu-se o tratamento de dados do dataset de House Prices, com destaque para as seguintes decisões:

```
# LotFrontage = c1·LotConfig + c2·sqrt(LotArea) + c3
df_lot = df_train[["LotArea", "LotConfig", "LotFrontage"]].copy()

df_lot = df_lot.dropna()

min_area = df_lot["LotArea"].min()
max_area = df_lot["LotArea"].max()

df_lot["LotArea"] = (df_lot["LotArea"] - min_area) / (max_area - min_area)
df_lot["sqrt_LotArea"] = np.sqrt(df_lot["LotArea"])
```


SEMANA 5 (PRÁTICA)

```
# Valores faltantes
df_nulos = df_train[df_train["MasVnrType"].isna() | df_train["MasVnrArea"].isna()]
print(df_nulos[["MasVnrType", "MasVnrArea"]])
```

	MasVnrType	MasVnrArea
234	NaN	NaN
529	NaN	NaN
624	NaN	288.0
650	NaN	NaN
773	NaN	1.0
936	NaN	NaN
973	NaN	NaN
977	NaN	NaN
1230	NaN	1.0
1243	NaN	NaN
1278	NaN	NaN
1300	NaN	344.0
1334	NaN	312.0

SEMANA 5 (PRÁTICA)

```
print(df_train.groupby("MasVnrType")["MasVnrArea"].agg(["mean", "std"]))

print(df_train["MasVnrType"].mode()[0])
```

	mean	std
MasVnrType		
BrkCmn	247.666667	182.423239
BrkFace	259.592342	215.027938
No masonry	0.000000	0.000000
Stone	241.188976	171.480469
No masonry		

```
# BrkFace é a categoria mais condizente para essas áreas entre 288 e 312
df_train.loc[[624, 1300, 1334], 'MasVnrType'] = 'BrkFace'

# Como "No masonry" é o mais comum, os outros valores ausentes ficaram assim
df_train["MasVnrType"] = df_train["MasVnrType"].fillna("No masonry")

# O resto das áreas assumiu valor zero
df_train["MasVnrArea"] = np.where(df_train["MasVnrType"] == "No masonry", 0, df_train["MasVnrArea"])
```

SEMANA 5 (PRÁTICA)

```
# Calculando quantidade
q1 = df_train["SalePrice"].quantile(0.25)
q3 = df_train["SalePrice"].quantile(0.75)
iqr = q3 - q1

li = q1 - 1.5 * iqr
ls = q3 + 1.5 * iqr

outliers = df_train[(df_train["SalePrice"] >= ls) | (df_train["SalePrice"] <= li)]

# Valor maior que 10%, melhor não tirar tudo
print(f"outliers:{len(outliers)}")

outliers = df_train[df_train["SalePrice"] > 500000]

# Tirando só os valores maiores que 500000
df_train = df_train.drop(outliers.index)
```

SEMANA 5 (PRÁTICA)

```
# Colunas de strings com 10 ou menos elementos únicos sofrem one hot encoding
onehot_cols = [
    col for col in cat_cols
    if x_train[col].nunique() <= 10
]

# Colunas de strings com mais de 10 únicos sofrem target encoding
target_cols = [
    col for col in cat_cols
    if x_train[col].nunique() > 10
]
```

SEMANA 5 (PRÁTICA)

```
# Treinamento
model.fit(x_train, y_train)

# Resultado
score = model.score(x_val, y_val)
print(score)
```

0.8608070642103068

S E M K - F O L D

5 - F O L D

```
# Resultados
print("R² médio:", scores.mean())
print("Desvio padrão:", scores.std())
```

R² médio: 0.7654850855529131
Desvio padrão: 0.12579354534196038

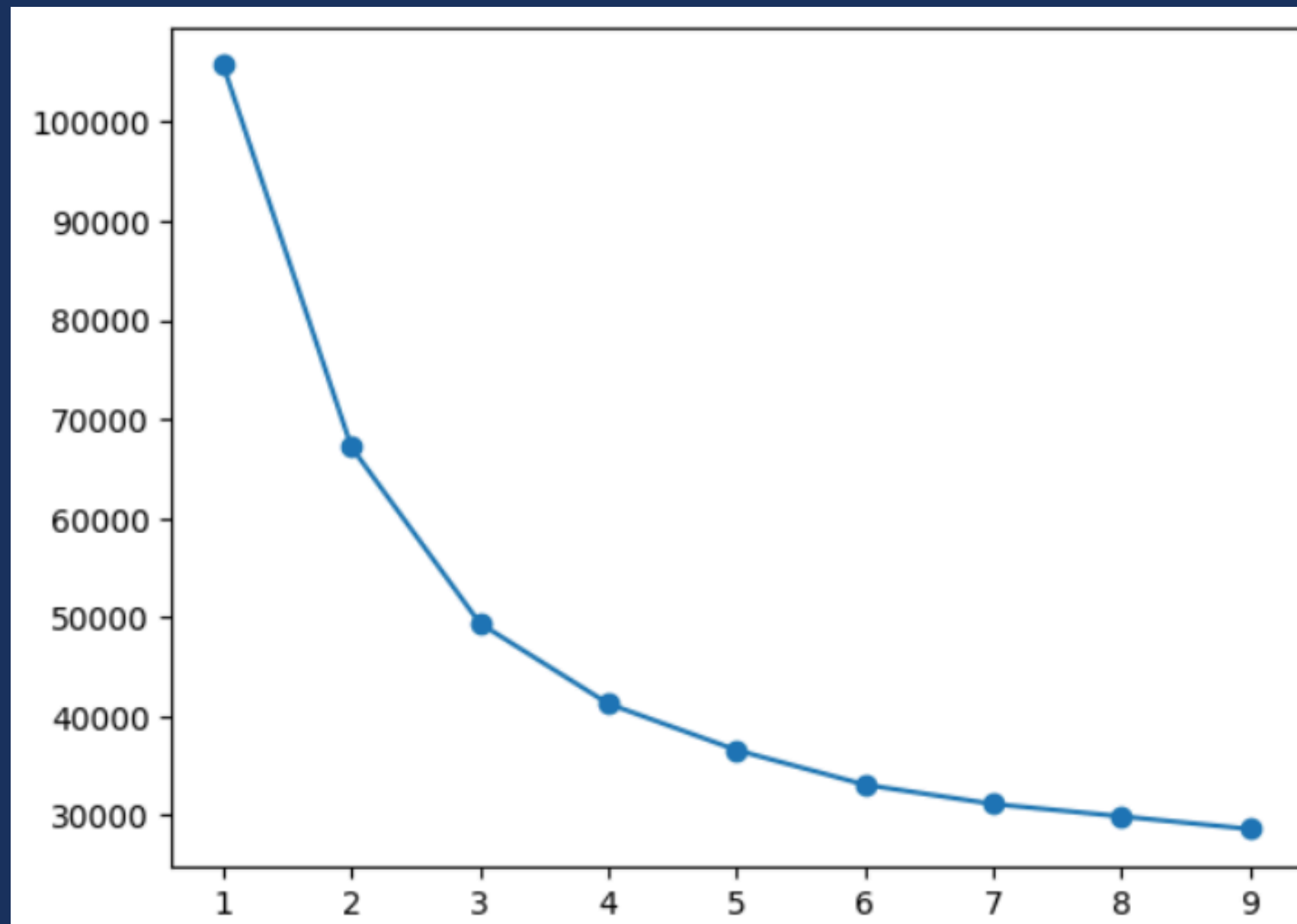
SEMANA 6

Nesta semana foram testados os algoritmos K-Means, KNN e MPLRegressor. Além do mais, foi implementado o streamlit, permitindo aos usuários preverem o valor de uma casa ao fornecerem algumas informações.

K-MEANS

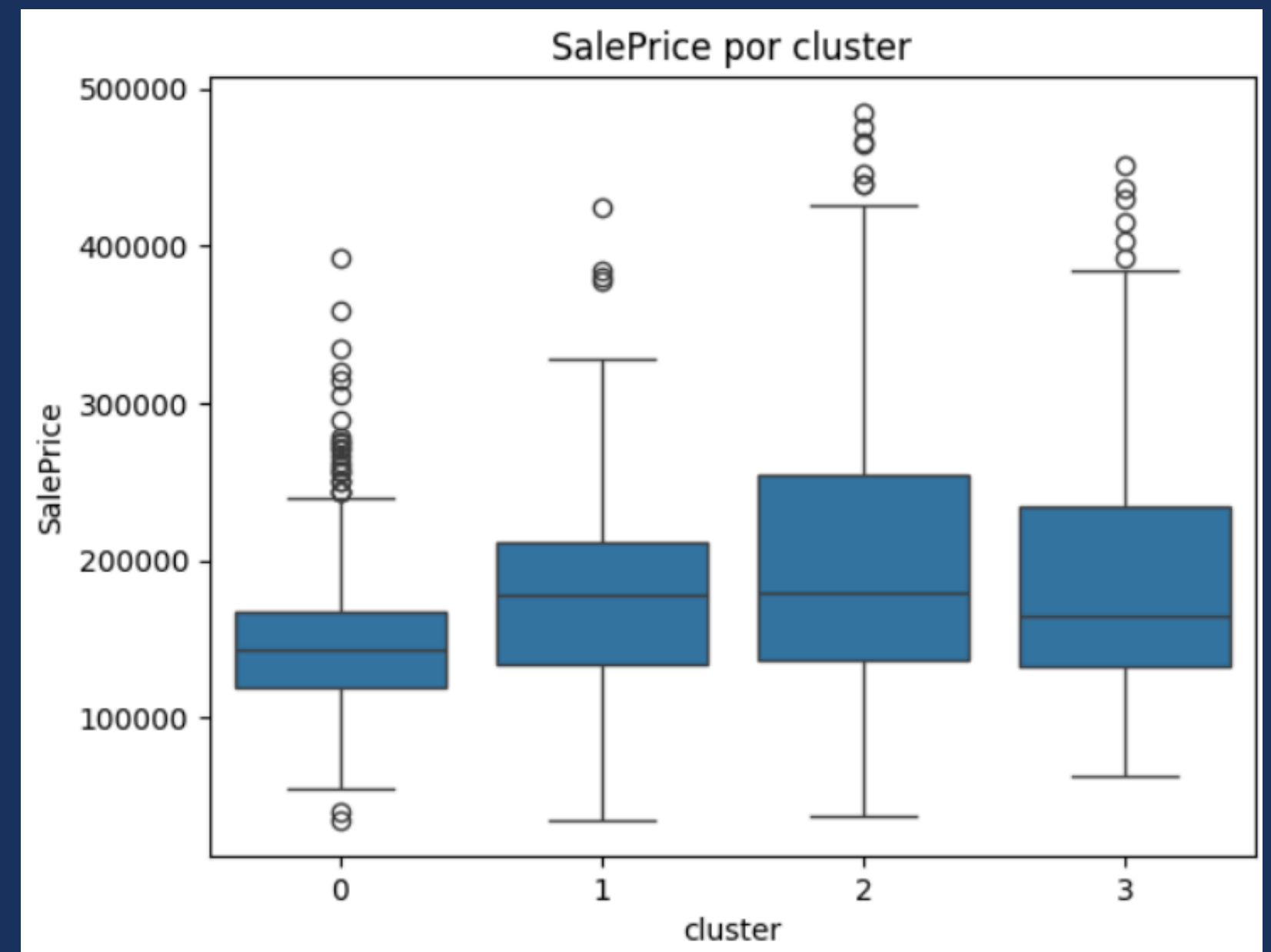
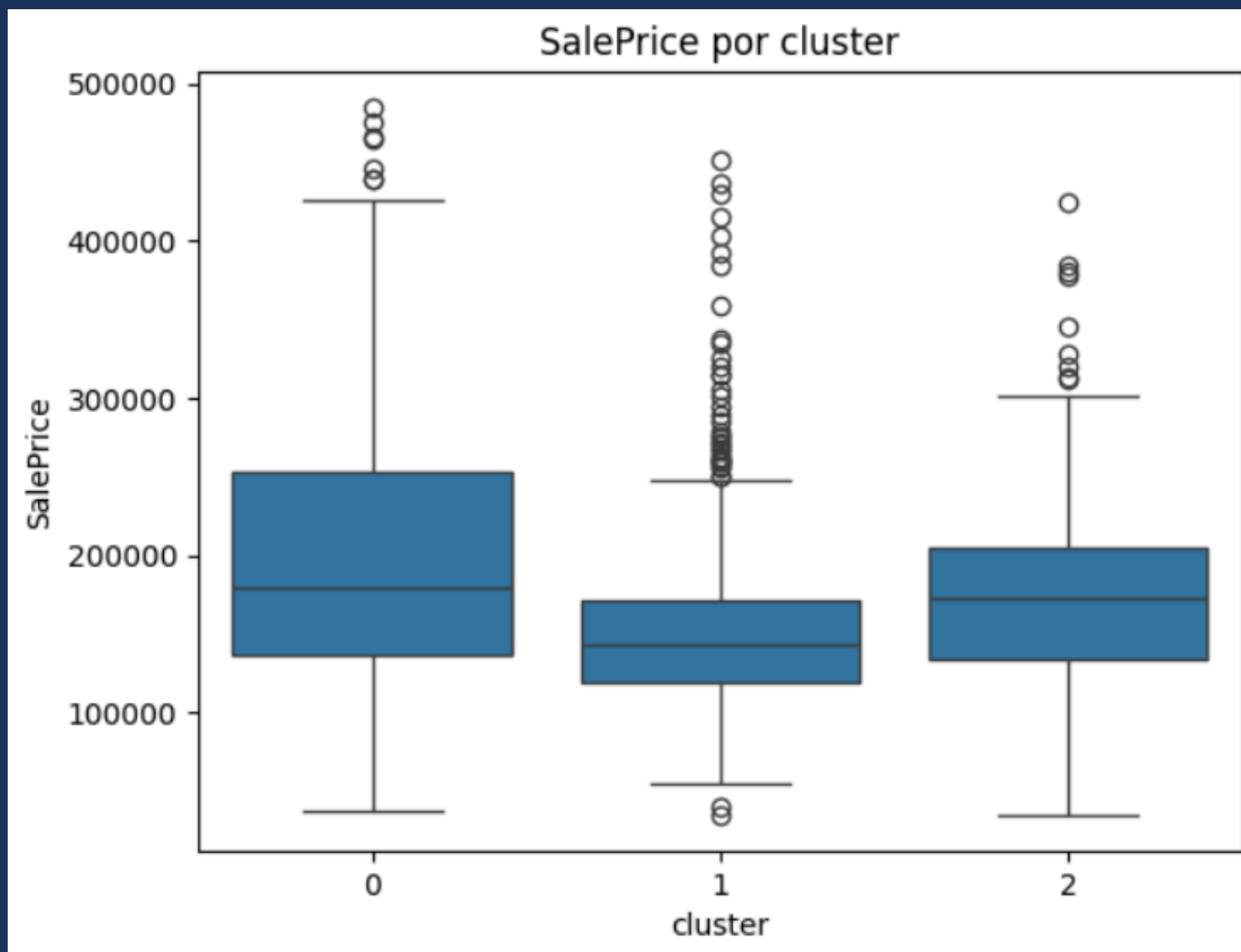
```
preprocess = ColumnTransformer(  
    transformers=[  
        ("onehot",  
         OneHotEncoder(handle_unknown="ignore"),  
         onehot_cols),  
  
        ("ordinal",  
         OrdinalEncoder(handle_unknown="use_encoded_value",  
                         unknown_value=-1),  
         target_cols), # Não dá para usar target-encoding em casos não supervisionados  
  
        ("num",  
         MinMaxScaler(),  
         num_cols)  
    ]  
)
```

K-MEANS



QUAL É O COTOVELO?

K-MEANS



KNN

```
# Determinando os intervalos  
df_train["SalePrice"].describe()
```

SalePrice	
count	1458.000000
mean	180140.566529
std	76645.846478
min	34900.000000
25%	129925.000000
50%	163000.000000
75%	213875.000000
max	625000.000000

KNN

```
def categoria(preco):  
    if preco <= 129900:  
        return "Low"  
    elif preco <= 162500:  
        return "Low-Average"  
    elif preco <= 213000:  
        return "High-Average"  
    else:  
        return "High"
```

```
def categoria(preco):  
    if preco <= 129900:  
        return "Low"  
    elif preco <= 213000:  
        return "Average"  
    else:  
        return "High"
```

4 CLASSES

K = 13, ACURÁCIA = 78%

3 CLASSES

K = 3, ACURÁCIA = 81%

MPLREGRESSOR

```
"regressor",  
MLPRegressor(  
    hidden_layer_sizes=(32),  
    activation="relu",  
    solver="adam",  
    max_iter=1000,  
    random_state=42  
)
```

R² MÉDIO: 0.508
DESVIO PADRÃO: 0.0484

```
"regressor",  
MLPRegressor(  
    hidden_layer_sizes=(32,32),  
    activation="relu",  
    solver="adam",  
    max_iter=1000,  
    random_state=42  
)
```

R² MÉDIO: 0.511
DESVIO PADRÃO: 0.0475

MPLREGRESSOR

```
"regressor",  
MLPRegressor(  
    hidden_layer_sizes=(32,32,32),  
    activation="relu",  
    solver="adam",  
    max_iter=1000,  
    random_state=42  
)
```

R² MÉDIO: 0.5085
DESVIO PADRÃO: 0.0497

```
"regressor",  
MLPRegressor(  
    hidden_layer_sizes=(8,8),  
    activation="relu",  
    solver="adam",  
    max_iter=1000,  
    random_state=42  
)
```

R² MÉDIO: 0.503
DESVIO PADRÃO: 0.0515

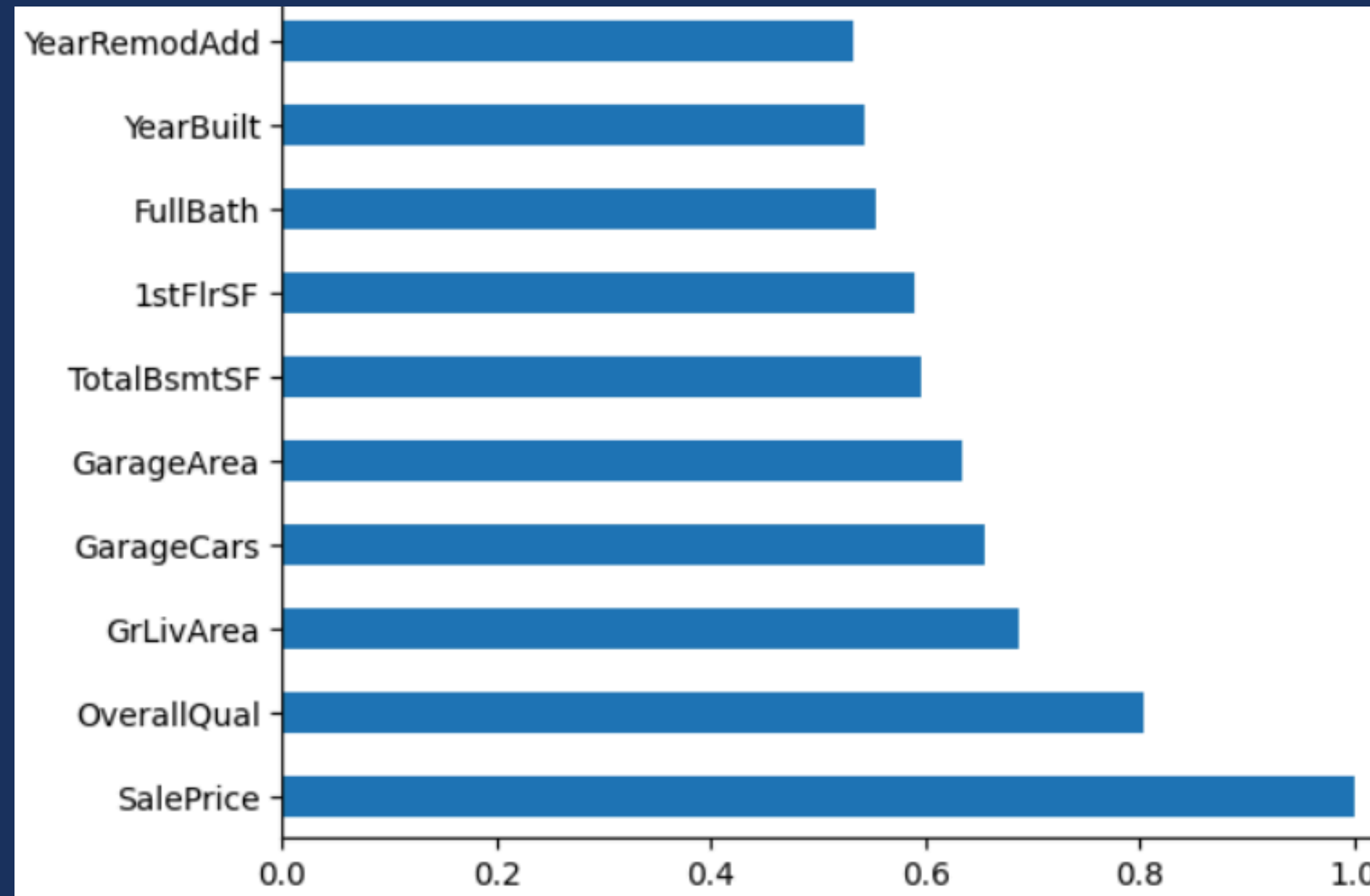
STREAMLIT

OVERALLQUAL

GrLivArea

3 MAIS IMPORTANTES!

GarageCars



STREAMLIT

Previsão de Preço

Área de vivência (em pés)

500,00

–

+

Quantia de carros que cabem na garagem

1,00

–

+

Qualidade da casa

5

Prever

Preço previsto: R\$ 91,143.48

SEMANA 7

Na última semana, foram implementados os algoritmos de RandomForestRegressor e GradientBoostingRegressor. Além do mais, fez-se um ensemble dos melhores modelos de Machine Learning do projeto.

Por fim, o tratamento de outlier foi alterado para analisar possíveis melhores resultados.

RANDOMFORESTREGRESSOR

```
for n in range(50, 501, 50):  
  
    modelo = Pipeline(  
        steps=[  
            ("preprocess", preprocess),  
            ("regressor", RandomForestRegressor(  
                n_estimators=n,  
                random_state=42  
            ))  
        ]  
    )
```

N_ESTIMATORS = 200
R² MÉDIO: 0.868
DESVIO PADRÃO: 0.024

GRADIENTBOOSTINGREGRESSOR

```
modelo = Pipeline(  
    steps=[  
        ("preprocess", preprocess),  
        ("regressor", GradientBoostingRegressor(  
            random_state=42  
        ))  
    ]  
)
```

R² MÉDIO: 0.877

DESVIO PADRÃO: 0.0399

ENSEMBLE

```
ensemble = y_pred_1 * 0.2 + y_pred_2 * 0.40 + y_pred_3 * 0.40  
  
print("Ensemble R²:", r2_score(y_val, ensemble))  
print("Ensemble mse:", mean_squared_error(y_val, ensemble))
```

ENSEMBLE R²: 0.903

ENSEMBLE MSE: 468213205.3080778

$\text{SQRT}(468213205.3080778) = \$21,638$

RESULTADOS

COMPARAÇÃO DOS MODELOS

Tratamento dos Outliers	Linear Regression (R ² ± DP)	Random Forest (R ² ± DP)	Gradient Boosting (R ² ± DP)
Sem remoção	0.715 ± 0.171	0.917 ± 0.011	0.931 ± 0.007
Remoção > 700000	0.809 ± 0.113	0.910 ± 0.020	0.919 ± 0.017
Remoção > 500000	0.765 ± 0.125	0.868 ± 0.024	0.877 ± 0.040
Remoção de todos os outliers	0.785 ± 0.121	0.924 ± 0.005	0.926 ± 0.008

RESULTADOS DO ENSEMBLE

Tratamento dos Outliers	R ²	MSE	RMSE
Sem remoção	0.939	404.397.464	20.110
Remoção > 700000	0.932	358.910.102	18.945
Remoção > 500000	0.903	468.213.205	21.638
Remoção de todos os outliers	0.924	269.250.694	16.409

TRATANDO DADOS DE TESTE

```
df_test["MSZoning"] = df_test["MSZoning"].fillna(df_test["MSZoning"].mode()[0])

df_test["Alley"] = df_test["Alley"].fillna("No alley access")

df_test["Utilities"] = df_test["Utilities"].fillna(df_test["Utilities"].mode()[0])
df_test["Exterior1st"] = df_test["Exterior1st"].fillna(df_test["Exterior1st"].mode()[0])
df_test["Exterior2nd"] = df_test["Exterior2nd"].fillna(df_test["Exterior2nd"].mode()[0])

df_test["MasVnrType"] = np.where(df_test["MasVnrArea"] == 0, "No masonry", df_test["MasVnrType"])

df_test["BsmtQual"] = df_test["BsmtQual"].fillna("No basement")
df_test["BsmtCond"] = df_test["BsmtCond"].fillna("No basement")
df_test["BsmtExposure"] = df_test["BsmtExposure"].fillna("No basement")
df_test["BsmtFinType1"] = df_test["BsmtFinType1"].fillna("No basement")
df_test["BsmtFinType2"] = df_test["BsmtFinType2"].fillna("No basement")
df_test["BsmtFinSF1"] = df_test["BsmtFinSF1"].fillna(0)
df_test["BsmtFinSF2"] = df_test["BsmtFinSF2"].fillna(0)
df_test["BsmtUnfSF"] = df_test["BsmtUnfSF"].fillna(0)
df_test["TotalBsmtSF"] = df_test["TotalBsmtSF"].fillna(0)
df_test["BsmtFullBath"] = df_test["BsmtFullBath"].fillna(0)
df_test["BsmtHalfBath"] = df_test["BsmtHalfBath"].fillna(0)

df_test["KitchenQual"] = df_test["KitchenQual"].fillna(df_test["KitchenQual"].mode()[0])
```


TRATANDO DADOS DE TESTE

```
df_nulos = df_test[df_train["MasVnrType"].isna() | df_test["MasVnrArea"].isna()]  
print(df_nulos[["MasVnrType", "MasVnrArea"]])
```

```
...      MasVnrType  MasVnrArea  
231          NaN          NaN  
246          NaN          NaN  
422          NaN          NaN  
532          NaN          NaN  
544          NaN          NaN  
581          NaN          NaN  
851          NaN          NaN  
865          NaN          NaN  
880          NaN          NaN  
889          NaN          NaN  
908          NaN          NaN  
1132         NaN          NaN  
1197         NaN          NaN  
1226         NaN          NaN  
1402         NaN          NaN
```

```
/tmp/ipykernel_1454/3112321513.py:1: UserWarning: Boolean Series key will be reindexed to match DataFrame index.  
df_nulos = df_test[df_train["MasVnrType"].isna() | df_test["MasVnrArea"].isna()]
```

```
# Sabendo que No masonry é o mais comum, segue o tratamento final  
df_test["MasVnrType"] = df_test["MasVnrType"].fillna("No masonry")  
df_test["MasVnrArea"] = df_test["MasVnrArea"].fillna(0)
```

CLASSIFICAÇÃO FINAL

2402

Eduardo Sanzovo Quaggio



0.13283



Your Best Entry!

Your most recent submission scored 0.13351, which is not an improvement of your previous score

OBRIGADO!

Dúvidas?

